Uji Koneksi Database Java Menggunakan JDBC

Analisis struktur proyek, kode Java, konfigurasi, SQL, tahapan pengujian, screenshot, diagnosis error, dan pembahasan laporan



D

I

S

U

S

U

N

Oleh:

Nama: Fhanny Jessica Octavia Sidauruk

NIM: 25110147

Mata kuliah: Pemprograman Klien Server Neatbens

Prodi: Teknik Komputer C

YAYASAN MINAR SORTA TIGA PUTRI

POLITEKNIK BISNIS INDONESIA

2026-2027

1. Tujuan Praktikum

Praktikum ini bertujuan memahami bagaimana aplikasi Java berkomunikasi dengan database MySQL atau MariaDB menggunakan JDBC (Java Database Connectivity).

Secara khusus, praktikum menguji kemampuan untuk:

- Membaca konfigurasi database dari file eksternal.
- Memuat driver JDBC agar Java mengenali MySQL.
- Membuka koneksi ke server database.
- Menjalankan query SQL untuk memeriksa tabel dan data.
- Menutup koneksi dengan benar.
- Mendiagnosis kesalahan koneksi berdasarkan pesan error.

Dengan demikian, praktikum tidak hanya menguji apakah koneksi berhasil, tetapi juga apakah program dapat memberikan informasi yang berguna ketika terjadi kegagalan.

2. Struktur dan Fungsi Seluruh File Penting

Struktur proyek yang relevan di dalam ZIP adalah sebagai berikut:

```
praktikum6/
├── bahan/
│   ├── db_inventori_lab.sql
│   └── mysql-connector-j-26.7.0/
└── proyek/PrKoneksiInventori/
    ├── src/koneksi/
    │   ├── KoneksiDB.java
    │   └── UjiKoneksi.java
    ├── config/
    │   ├── database.properties
    │   └── database.example.properties
    ├── Lib/
    └── nbproject/
```

bahan/: berisi SQL untuk data awal database inventori dan distribusi/library MySQL Connector/J.

proyek/PrKoneksiInventori/: proyek Java yang dibuka melalui NetBeans.

src/koneksi/KoneksiDB.java: membaca konfigurasi dan menyediakan koneksi JDBC.

src/koneksi/UjiKoneksi.java: menjalankan pengujian bertahap dan melaporkan error.

config/database.properties: konfigurasi koneksi yang dibaca saat program dijalankan.

config/database.example.properties: contoh konfigurasi untuk disalin dan disesuaikan.

Lib/: library yang dibutuhkan proyek, termasuk file JAR JDBC.

nbproject/: pengaturan proyek NetBeans, classpath, versi Java, dan kelas utama.

Screenshot pengujian: empat skenario yang diberi nama hasil uji, MySQL mati, password salah, dan driver hilang. Salinan gambar juga terdapat pada subfolder lain di ZIP.

Fungsi file tambahan

- project.properties: mengatur classpath, versi sumber Java, dan kelas utama koneksi.UjiKoneksi.
- project.xml: menyimpan informasi identitas dan konfigurasi proyek NetBeans.
- build-impl.xml: mendukung proses build otomatis yang dihasilkan NetBeans.
- manifest.mf: metadata untuk file JAR hasil build.
- built-jar.properties: informasi hasil build sebelumnya, bukan bukti bahwa pengujian saat ini sukses.
- mysql-connector-j-26.7.0.jar: library penghubung Java dengan MySQL. Folder sumber, dokumentasi, dan pengujian dalam distribusi Connector/J merupakan bagian dari paket library, bukan kode utama aplikasi inventori.

3. Penjelasan File KoneksiDB.java

File ini merupakan kelas yang bertanggung jawab mengelola koneksi database. Kelas ini memisahkan pengaturan koneksi dari kode program pengujian.

A. Package dan import

```
package koneksi;

import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;
```

- package koneksi; menempatkan kelas dalam paket koneksi.
- Files, Path, dan Paths digunakan untuk mencari serta membaca file konfigurasi.
- InputStream digunakan untuk membaca isi file.
- Connection mewakili koneksi aktif ke database.
- DriverManager membantu membuat koneksi JDBC.
- SQLException menangani kesalahan operasi database.
- Properties menyimpan pasangan nama properti dan nilainya.

B. Menentukan driver dan lokasi konfigurasi

```
public static final String DRIVER =
        "com.mysql.cj.jdbc.Driver";

public static final Path FILE_KONFIGURASI =
        Paths.get("config", "database.properties");
```

DRIVER berisi nama kelas driver MySQL Connector/J. FILE_KONFIGURASI menunjukkan bahwa program mencari file konfigurasi di folder config, relatif terhadap direktori kerja saat program dijalankan. Karena itu, proyek sebaiknya dijalankan dengan direktori kerja yang sesuai, biasanya folder utama PrKoneksiInventori.

C. Menentukan konfigurasi wajib

```
private static final String[] KUNCI_WAJIB = {
    "db.host", "db.port", "db.name", "db.user"
};
```

Keempat properti ini wajib memiliki nilai:

Properti	Fungsi
db.host	Alamat server database, misalnya 127.0.0.1
db.port	Port server, biasanya 3306
db.name	Nama database yang akan digunakan
db.user	Username untuk autentikasi database

Password dibaca terpisah dan tidak termasuk dalam daftar properti wajib. Karena itu, password kosong belum tentu membuat tahap pembacaan konfigurasi gagal, tetapi autentikasi dapat gagal ketika koneksi dibuka.

D. Membaca dan memvalidasi konfigurasi

```
public static Properties bacaKonfigurasi()
        throws SQLException {
    Path lokasi = FILE_KONFIGURASI.toAbsolutePath();
    Properties konfigurasi = new Properties();

    try (InputStream input = Files.newInputStream(lokasi)) {
        konfigurasi.load(input);
    } catch (IOException e) {
        throw new SQLException(
                "File konfigurasi tidak ditemukan atau tidak dapat dibaca: "
                + lokasi, e);
    }
```

Kode mengubah lokasi konfigurasi menjadi path absolut, membuka file, lalu membaca isinya ke objek Properties. Kegagalan membaca file diubah menjadi SQLException dengan pesan yang menjelaskan masalah. try-with-resources memastikan InputStream ditutup otomatis setelah pembacaan.

```
for (String kunci : KUNCI_WAJIB) {
    if (konfigurasi.getProperty(kunci, "")
            .trim().isEmpty()) {
        throw new SQLException(
                "Nilai " + kunci
                + " wajib diisi pada file " + lokasi);
    }
}
```

getProperty() mengambil nilai properti. Jika tidak ada, nilai penggantinya string kosong. trim() membuang spasi di awal dan akhir. Jika hasilnya kosong, program melempar exception.

```
String port = konfigurasi.getProperty("db.port").trim();

if (!port.matches("\\d{1,5}")) {
    throw new SQLException(
            "Nilai db.port harus berupa angka, contoh 3306. Isi saat ini: "
            + port);
}
```

Regex \\d{1,5} berarti nilai port harus terdiri atas 1 sampai 5 digit. Validasi ini memeriksa format angka, tetapi tidak memeriksa apakah nilainya berada pada rentang port yang sah, yaitu 1–65535.

E. Membentuk URL JDBC

```
public static String buatUrl(Properties konfigurasi) {
    return "jdbc:mysql://"
            + konfigurasi.getProperty("db.host").trim() + ":"
            + konfigurasi.getProperty("db.port").trim() + "/"
            + konfigurasi.getProperty("db.name").trim()
            + "?connectTimeout=5000&socketTimeout=10000";
}
```

URL menjadi alamat yang digunakan driver JDBC untuk mengetahui server dan database tujuan.

- jdbc:mysql://: protokol koneksi JDBC untuk MySQL.
- 127.0.0.1: alamat server lokal.
- 3306: port database.
- db_inventori_lab: nama database.
- connectTimeout=5000: batas waktu sekitar 5.000 milidetik untuk membuat koneksi.
- socketTimeout=10000: batas waktu sekitar 10.000 milidetik untuk menunggu operasi baca pada socket.

Timeout membantu mencegah program menunggu tanpa batas waktu ketika server tidak merespons.

F. Memuat driver JDBC

```
public static void muatDriver() throws SQLException {
    try {
        Class.forName(DRIVER);
    } catch (ClassNotFoundException e) {
        throw new SQLException(
                "Driver JDBC " + DRIVER + " tidak ditemukan. "
                + "Tambahkan mysql-connector-j-26.7.0.jar ke Libraries proyek.",
                e);
    }
}
```

Class.forName() meminta Java memuat kelas driver. Jika JAR tidak ada di classpath, Java menghasilkan ClassNotFoundException. Kode menangkap kesalahan tersebut dan mengubahnya menjadi pesan yang lebih mudah dipahami. Keberhasilan memuat driver hanya membuktikan bahwa kelas driver tersedia; hal itu belum membuktikan server MySQL hidup atau kredensial benar.

G. Membuka koneksi

```
public static Connection getConnection() throws SQLException {
    Properties konfigurasi = bacaKonfigurasi();
    muatDriver();

    return DriverManager.getConnection(
            buatUrl(konfigurasi),
            konfigurasi.getProperty("db.user").trim(),
            konfigurasi.getProperty("db.password", ""));
}
```

Urutannya: membaca konfigurasi, memastikan driver tersedia, membentuk URL koneksi, mengirim username dan password ke DriverManager, lalu mengembalikan objek Connection jika koneksi berhasil. Jika database tidak ditemukan atau password salah, JDBC melempar SQLException.

Kesimpulan KoneksiDB.java: kelas ini berfungsi sebagai pusat pengaturan koneksi. Kode lebih mudah dirawat karena konfigurasi, pemuatan driver, dan pembukaan koneksi tidak perlu ditulis ulang di setiap program.

4. Penjelasan File UjiKoneksi.java

File ini adalah program utama yang menjalankan seluruh tahap pengujian. Kelas utama proyek diatur sebagai koneksi.UjiKoneksi sehingga dapat dijalankan langsung dari NetBeans.

A. Import dan fungsi utama

Program menggunakan Connection untuk koneksi database, DatabaseMetaData untuk informasi server dan driver, PreparedStatement untuk menyiapkan SQL, ResultSet untuk menampung hasil query, SQLException untuk kesalahan database, serta UnknownHostException, ConnectException, dan SocketTimeoutException untuk membantu mengidentifikasi gangguan koneksi.

```
public static void main(String[] args) {
    int tahap = 0;
    Connection koneksiTerpakai = null;

    System.out.println("=== UJI KONEKSI JDBC - PRAKTIKUM 6 ===");
```

Variabel tahap menyimpan nomor tahapan yang sedang dikerjakan. Variabel koneksiTerpakai menyimpan referensi koneksi untuk memeriksa apakah koneksi telah ditutup.

B. Tahap 1 — Membaca file konfigurasi

```
tahap = 1;
System.out.println("[TAHAP 1] Membaca file konfigurasi");

Properties konfigurasi = KoneksiDB.bacaKonfigurasi();
```

Program memanggil bacaKonfigurasi() dari KoneksiDB. Setelah berhasil, program menampilkan lokasi absolut file konfigurasi, URL JDBC, username database, dan status password tanpa menampilkan nilainya.

```
[TAHAP 1] Membaca file konfigurasi
File     : ...\\config\\database.properties
URL      : jdbc:mysql://127.0.0.1:3306/db_inventori_lab...
User     : lab_p06
Password : (terisi, tidak ditampilkan)
-> BERHASIL
```

Tujuannya memastikan konfigurasi dapat dibaca dan nilai yang diperlukan tersedia. Tahap ini belum menghubungi database, sehingga tahap 1 bisa berhasil walaupun database tidak aktif atau namanya salah.

C. Tahap 2 — Memuat driver JDBC

```
tahap = 2;
System.out.println("[TAHAP 2] Memuat driver JDBC");

KoneksiDB.muatDriver();
```

```
System.out.println("Driver : " + KoneksiDB.DRIVER);
System.out.println("-> BERHASIL");
```

Program memanggil muatDriver() untuk memuat kelas driver MySQL. Jika berhasil, output menampilkan com.mysql.cj.jdbc.Driver. Tahap ini memastikan library tersedia, bukan memastikan server hidup atau password benar.

D. Tahap 3 — Membuka koneksi ke server database

```
tahap = 3;
System.out.println(
        "[TAHAP 3] Membuka koneksi ke server database");

long mulai = System.currentTimeMillis();

try (Connection con = KoneksiDB.getConnection()) {
    koneksiTerpakai = con;

    long durasi = System.currentTimeMillis() - mulai;
    DatabaseMetaData meta = con.getMetaData();
```

Tahap ini memanggil getConnection() untuk menghubungkan Java ke server database. Waktu sebelum dan sesudah pembukaan koneksi dicatat untuk memperoleh durasi koneksi. Jika berhasil, program mengambil informasi server melalui DatabaseMetaData dan menampilkan durasi koneksi, nama dan versi server, nama dan versi driver, nama database, serta akun yang terhubung.

Tahap 3 menguji apakah server dapat dijangkau, database tersedia, akun dapat melakukan autentikasi, dan koneksi JDBC dapat dibuka. Jika gagal, penyebabnya bisa berupa server mati, database tidak ditemukan, password salah, atau hak akses yang tidak sesuai.

E. Tahap 4 — Menjalankan query verifikasi

```
tahap = 4;
System.out.println("[TAHAP 4] Menjalankan query verifikasi");

String sql = "SELECT COUNT(*) AS jumlah FROM barang";

try (PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
    rs.next();
    System.out.println("Query : " + sql);
    System.out.println(
            "Hasil : " + rs.getInt("jumlah") + " barang");
}
System.out.println("-> BERHASIL");
```

SELECT COUNT(*) menghitung jumlah baris, AS jumlah memberi nama pada kolom hasil, dan FROM barang menentukan tabel yang dihitung. PreparedStatement menyiapkan SQL, executeQuery() menjalankannya, dan ResultSet menampung hasil.

rs.next() memindahkan kursor ke baris hasil pertama. getInt("jumlah") mengambil hasil sebagai bilangan bulat.

```
```

```
[TAHAP 4] Menjalankan query verifikasi
Query : SELECT COUNT(*) AS jumlah FROM barang
Hasil : 8 barang
-> BERHASIL
```

Angka delapan hanyalah contoh. Jumlah aktual bergantung pada isi database. Query menghitung seluruh baris tabel, bukan menjumlahkan nilai stok.

Tahap ini membuktikan lebih dari sekadar koneksi aktif: program juga dapat menjalankan query dan membaca hasilnya.

F. Tahap 5 — Menutup koneksi

```
try (Connection con = KoneksiDB.getConnection()) {
    // Pengujian koneksi dan query
}
```

Ketika blok try-with-resources selesai, Java menutup koneksi secara otomatis.

```
tahap = 5;
System.out.println("[TAHAP 5] Menutup koneksi");

System.out.println(
        "Koneksi sudah ditutup? " + koneksiTerpakai.isClosed());

System.out.println("-> BERHASIL");
```

Jika koneksi sudah ditutup, isClosed() menghasilkan true. Tujuannya mencegah koneksi dibiarkan terbuka dan memastikan sumber daya server dapat digunakan kembali.

G. Pesan keberhasilan seluruh tahap

```
System.out.println("=== SEMUA TAHAP BERHASIL ===");
```

Pesan ini hanya tercapai apabila seluruh proses dari pembacaan konfigurasi hingga penutupan koneksi selesai tanpa SQLException. Jika salah satu tahap gagal, program masuk ke bagian penanganan kesalahan dan pesan akhir keberhasilan tidak ditampilkan.

H. Penanganan kesalahan dengan catch

```
} catch (SQLException e) {
    laporkanGagal(tahap, e);
}
```

Jika terjadi SQLException, program memanggil laporkanGagal(). Metode ini menampilkan tahap yang gagal, SQLState, kode error, pesan kesalahan, penyebab dasar, dan saran perbaikan. Blok catch utama hanya menangani SQLException; exception lain yang tidak dikonversi menjadi SQLException belum tentu mengikuti format laporan error yang sama.

5. Penjelasan File SQL db_inventori_lab.sql

File SQL menyediakan data awal bagi database inventori laboratorium. File ini bukan kode Java, melainkan kumpulan perintah SQL yang dijalankan pada server database.

A. Pengaturan awal

```
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
SET AUTOCOMMIT = 0;
START TRANSACTION;
SET time_zone = "+00:00";
```

SET SQL_MODE menentukan perilaku tertentu server SQL. SET AUTOCOMMIT = 0 menonaktifkan commit otomatis. START TRANSACTION memulai transaksi. SET time_zone menentukan zona waktu sesi SQL. Pada akhir file terdapat COMMIT untuk mengesahkan perubahan transaksi.

B. Data tabel barang

File berisi delapan perintah data barang, antara lain keyboard USB, mouse USB, kabel HDMI, kabel UTP, switch, access point, multimeter, dan LAN tester. Kolomnya meliputi:

Kolom	Fungsi
kode_barang	Identitas kode barang
nama_barang	Nama barang
id_kategori	Referensi kategori
jumlah_tersedia	Jumlah stok yang tersedia
keterangan	Lokasi atau catatan barang

Data ini menjadi dasar pengujian tahap 4. COUNT(*) menghitung seluruh baris pada tabel, bukan menjumlahkan stok. Jika delapan jenis barang dicatat sebagai delapan baris, hasilnya delapan meskipun jumlah stok tiap barang berbeda.

C. Data tabel kategori

File berisi empat kategori: Perangkat Input, Kabel dan Konektor, Perangkat Jaringan, dan Alat Ukur. id_kategori menghubungkan barang dengan kelompoknya; keyboard dan mouse misalnya menggunakan kategori perangkat input.

D. Data tabel peminjaman

File berisi tiga catatan transaksi peminjaman: PJM-0001 berstatus DIKEMBALIKAN, sedangkan PJM-0002 dan PJM-0003 berstatus DIPINJAM. Kolom tanggal_pinjam, tanggal_kembali, dan id_pengguna mencatat informasi transaksi. NULL pada tanggal kembali menunjukkan tanggal pengembalian belum diisi.

E. Data tabel detail_peminjaman

Tabel ini menyimpan barang dan jumlah barang pada setiap transaksi. id_peminjaman menghubungkan detail dengan transaksi, sedangkan kode_barang menghubungkannya dengan data barang.

F. Data tabel pengguna

File berisi dua akun contoh, administrator dan petugas laboratorium. Kolomnya mencakup username, password_hash, nama lengkap, peran, status aktif, dan waktu pembuatan. Nilai password_hash dalam file merupakan placeholder BELUM_DIATUR_P14, bukan hash password siap pakai untuk autentikasi aplikasi.

G. Kekurangan penting file SQL

Berdasarkan pemeriksaan, file berisi INSERT untuk memasukkan data, tetapi tidak berisi CREATE TABLE. Database dan struktur tabel harus dibuat terlebih dahulu. Mengimpor file ke database kosong tanpa tabel yang sesuai akan menghasilkan error tabel tidak ditemukan. Urutan impor harus sesuai struktur tabel, tipe data, dan relasi foreign key.

6. Analisis File Konfigurasi Database

File database.properties mengatur alamat server, nama database, username, dan password. File ini digunakan oleh KoneksiDB.java agar pengaturan tidak ditulis langsung ke source code.

```
db.host=127.0.0.1
db.port=3306
db.name=db_inventori_lab
db.user=USERNAME_DATABASE
db.password=PASSWORD_LOKAL
```

127.0.0.1 berarti server database berada di komputer yang sama. Port 3306 umum digunakan MySQL. db_inventori_lab adalah nama database tujuan. Username dan password harus sesuai dengan akun yang dibuat di MySQL/MariaDB.

database.example.properties dapat dijadikan acuan. Salin dan sesuaikan nilainya ke database.properties. Password tersimpan dalam teks biasa harus dijaga dan tidak dibagikan dalam laporan, screenshot, atau repositori publik. Karena kredensial sudah masuk ke arsip yang dibagikan, sebaiknya ganti password akun lokal terkait sebelum digunakan kembali, terutama jika akun memiliki hak akses di luar database praktikum.

7. Analisis Screenshot Uji Coba

Empat screenshot di dalam ZIP menunjukkan satu pengujian normal yang diberi nama “berhasil” serta tiga skenario kegagalan. Nama file tidak selalu sesuai dengan hasil yang terlihat di jendela Output.

A. Screenshot 1 — P06_01_berhasil.png.png

Hasil aktual: gagal pada tahap 3. Output memperlihatkan tahap 1 dan 2 berhasil, tetapi tahap 3 gagal dengan pesan Unknown database 'db_inventori_lab', kode error 1049, dan SQLState 42000.

Analisis: konfigurasi berhasil dibaca dan driver berhasil dimuat. Namun, server database yang dihubungi tidak menemukan database dengan nama tersebut.

Perbaikan: pastikan database db_inventori_lab dibuat pada server MySQL/MariaDB yang berjalan, lalu periksa db.name pada file konfigurasi. Screenshot ini belum membuktikan seluruh tahap koneksi berhasil. Nama file perlu disesuaikan atau screenshot keberhasilan penuh diambil setelah masalah diperbaiki.

B. Screenshot 2 — P06_02_E1_mysql_mati.png.png

Output memperlihatkan Communications link failure, SQLState 08S01, dan penyebab ConnectException - Connection refused.

Analisis: Java berhasil memuat driver, tetapi koneksi TCP ke server database ditolak. Penyebab paling mungkin adalah layanan MySQL/MariaDB belum aktif atau port tidak sesuai.

1. Buka XAMPP Control Panel.
2. Jalankan MySQL jika belum aktif.
3. Periksa port yang digunakan server.

4. Pastikan db.port sesuai dengan port tersebut.
5. Jalankan ulang program.

Screenshot ini menunjukkan bahwa program dapat mendeteksi gangguan koneksi dan menampilkan saran perbaikan.

C. Screenshot 3 — P06_03_E6_password_salah.png.png

Output memperlihatkan Access denied for user, kode error 1045, dan SQLState 28000.

Analisis: server database berhasil dijangkau, tetapi autentikasi akun ditolak. Ini berbeda dari screenshot 2: server merespons permintaan koneksi, tetapi tidak menerima kredensial atau hak autentikasi akun tersebut.

6. Periksa username dan password pada database.properties.
7. Pastikan akun database tersedia.
8. Periksa apakah akun diizinkan terhubung dari host localhost.
9. Pastikan password sesuai dengan akun lokal.

Screenshot ini membuktikan bahwa program mampu membedakan kegagalan autentikasi dari kegagalan jaringan.

D. Screenshot 4 — P06_04_E10_driver_hilang.png.png

Output menunjukkan kelas driver com.mysql.cj.jdbc.Driver tidak ditemukan.

Analisis: tahap 1 berhasil, tetapi program tidak dapat memuat kelas driver pada tahap 2. Masalah berkaitan dengan library atau classpath Java, bukan nama database maupun password.

10. Buka proyek PrKoneksiInventori di NetBeans.
11. Buka bagian Libraries.
12. Pastikan mysql-connector-j-26.7.0.jar ditambahkan sebagai library proyek.
13. Periksa bahwa file JAR benar-benar tersedia.
14. Lakukan Clean and Build, kemudian jalankan ulang UjiKoneksi.java.

Screenshot ini menunjukkan bahwa program memiliki pemeriksaan khusus untuk mendeteksi driver yang hilang sebelum mencoba menghubungi server.

Catatan: gunakan gambar asli dari ZIP sebagai bukti visual utama di laporan. Analisis di atas didasarkan pada jenis output/error yang tercatat.

8. Perbandingan Hasil Empat Pengujian

Screenshot	Tahap gagal	Kode/error utama	Makna
1. berhasil	3	1049, 42000	Database tidak ditemukan
2. MySQL mati	3	08S01, koneksi ditolak	Server tidak menerima koneksi
3. Password salah	3	1045, 28000	Autentikasi ditolak
4. Driver hilang	2	ClassNotFoundException	Library JDBC tidak ditemukan

Keempat hasil menunjukkan program tidak hanya menjalankan koneksi, tetapi juga mengklasifikasikan sejumlah kondisi gagal berdasarkan informasi error.

Mengapa kode error penting?

- Error 1049: nama database tidak ditemukan.
- Error 1045: autentikasi akun ditolak.
- Error 1146: tabel yang diminta tidak ditemukan.
- Error 1142: akun tidak mempunyai izin menjalankan operasi SQL tertentu.
- SQLState 08S01: menunjukkan gangguan komunikasi.
- ClassNotFoundException: kelas driver tidak tersedia pada classpath.

Informasi ini membantu menentukan bagian mana yang harus diperbaiki tanpa harus menebak penyebabnya.

9. Cara Menjalankan Pengujian dari Awal

Langkah 1 — Siapkan database

Jalankan MySQL/MariaDB melalui XAMPP atau layanan database yang digunakan. Buat database db_inventori_lab dan buat struktur tabel yang dibutuhkan.

Langkah 2 — Impor data SQL

Jalankan db_inventori_lab.sql setelah tabel yang sesuai tersedia. Periksa apakah tabel barang, kategori, peminjaman, detail_peminjaman, dan pengguna sudah terisi sesuai kebutuhan.

Langkah 3 — Periksa konfigurasi

Sesuaikan host, port, nama database, username, dan password pada config/database.properties.

Langkah 4 — Periksa library

Pastikan mysql-connector-j-26.7.0.jar tersedia di Libraries proyek. Referensi JAR absolut Windows dapat perlu diperbaiki jika proyek dipindahkan ke komputer lain.

Langkah 5 — Jalankan program

Di NetBeans, buka src/koneksi/UjiKoneksi.java, klik kanan file dan pilih Run File atau gunakan Shift + F6.

Langkah 6 — Catat hasil

Periksa output tahap 1 sampai 5. Jika ada error, catat nomor tahap, SQLState, kode error, dan pesan penyebab sebelum melakukan perbaikan.

10. Pengujian Normal dan Pengujian Negatif

Pengujian normal (positive test)

Pengujian normal dilakukan dengan konfigurasi benar, server aktif, database tersedia, dan driver terpasang. Targetnya seluruh lima tahap berhasil dan pesan akhir === SEMUA TAHAP BERHASIL === muncul.

Pengujian negatif (negative test)

Pengujian negatif dilakukan dengan mensimulasikan masalah seperti server tidak aktif, password salah, atau driver tidak tersedia. Tujuannya bukan membuat program gagal tanpa alasan, melainkan memastikan program mengenali kesalahan dengan tepat dan memberikan saran perbaikan.

Tiga screenshot error dalam ZIP merupakan contoh pengujian negatif. Screenshot berlabel P06_01_berhasil justru menunjukkan database tidak ditemukan, sehingga belum dapat dianggap sebagai hasil pengujian normal yang sukses.

11. Evaluasi Kualitas Program

Secara struktur, kode menerapkan beberapa praktik yang baik:

- Pemisahan tanggung jawab: KoneksiDB menangani koneksi, sedangkan UjiKoneksi menjalankan pengujian.
- Konfigurasi eksternal: parameter koneksi tidak ditulis langsung ke source code.
- Penanganan error: program menampilkan pesan dan saran perbaikan berdasarkan jenis kegagalan.
- Manajemen sumber daya: try-with-resources membantu memastikan koneksi dan objek SQL ditutup.
- Verifikasi fungsional: query COUNT(*) menguji akses nyata terhadap tabel database.

Hal yang perlu diperhatikan:

- File SQL hanya memasukkan data; struktur tabel harus disiapkan terpisah.
- Password yang tersimpan dalam file konfigurasi biasa perlu dijaga agar tidak dibagikan.
- Referensi library absolut di konfigurasi NetBeans dapat menyulitkan pemindahan proyek.
- Validasi port dapat diperketat untuk memastikan nilai berada pada rentang yang sah.
- Pengujian baru dinyatakan sukses apabila seluruh tahap selesai tanpa error, bukan sekadar tahap 1 dan 2 berhasil.

12. Kesimpulan Akhir

Dari analisis kode, file konfigurasi, file SQL, dan screenshot di dalam ZIP, dapat disimpulkan bahwa proyek ini dirancang untuk menguji koneksi JDBC secara bertahap sekaligus mengidentifikasi beberapa masalah umum.

Kode telah memiliki struktur yang cukup baik untuk tujuan pembelajaran. Namun, bukti screenshot yang tersedia belum menunjukkan seluruh lima tahap berhasil. Tiga screenshot menggambarkan kegagalan yang disengaja atau disimulasikan, sedangkan screenshot berlabel berhasil justru menunjukkan database tidak ditemukan pada tahap 3.

Untuk memperoleh hasil akhir praktikum yang lengkap, perbaiki database dan konfigurasi terlebih dahulu, jalankan ulang program, kemudian ambil screenshot baru yang menampilkan seluruh lima tahap berhasil. Setelah itu, dokumentasikan pula screenshot pengujian negatif sebagai bukti bahwa mekanisme penanganan kesalahan berfungsi.